Multimodal Table Scanning Application

Note: The RDK X5 4GB version cannot run this offline due to performance limitations. Please refer to the tutorial for online large models.

1. Concept Introduction

1.1 What is "Multimodal Table Scanning"?

Multimodal table scanning is a technology that uses image processing and artificial intelligence to identify and extract table information from images or PDF documents. It not only focuses on the visual recognition of table structures but also combines multimodal data such as text content and layout information to enhance the understanding of tables. **Large Language Models (LLMs)** provide powerful semantic analysis capabilities for understanding this extracted information, and the two complement each other to jointly improve the intelligence level of document processing.

1.2 Brief Description of the Principle

1. **Table Detection and Content Recognition**
   - Use computer vision technology to locate tables in documents and convert the text within the tables into editable format through OCR technology.
   - Use deep learning methods to parse the table structure (row and column division, merged cells, etc.) to generate a structured data representation.
2. **Multimodal Fusion**
   - Integrate visual (such as table layout), textual (OCR results), and possibly existing metadata (such as file type, source) to form a comprehensive data view.
   - Use specially designed multimodal models (such as LayoutLM) to process these different types of data simultaneously in order to more accurately understand the table content and its contextual relationships.

2. Code Analysis

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `scan_table` function in this file defines the execution flow of the tool, especially how it constructs a Prompt that requests a return in Markdown format.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def scan_table(self, args):
        """
        Scan a table from an image and save the content as a Markdown file.

        :param args: Arguments containing the image path.
        :return: Dictionary with file path and content.
        """
        self.node.get_logger().info(f"Executing scan_table() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            # ... (Path checking and fallback)

            # Construct a prompt asking the large model to recognize the table
and return it in Markdown format.
            if self.node.language == 'zh':
                prompt = "请仔细分析这张图片，识别其中的表格，并将其内容以Markdown格式返
回。"
            else:
                prompt = "Please carefully analyze this image, identify the table
within it, and return its content in Markdown format."

            result = self.node.model_client.infer_with_image(image_path, prompt)

            # ... (Extract Markdown text from the result)

            # Save the recognized content to a Markdown file.
            md_file_path = os.path.join(self.node.pkg_path, "resources_file",
"scanned_tables", f"table_{timestamp}.md")
            with open(md_file_path, 'w', encoding='utf-8') as f:
                f.write(table_content)

            return {
                "file_path": md_file_path,
                "table_content": table_content
            }
        # ... (Error handling)
```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare message)
        try:
            # Decide which specific implementation to call based on the value of
self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages,
image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Logic for calling Tongyi model
                pass
            # ... (Logic for other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code Analysis

The table scanning function is a typical application of converting unstructured image data into structured text data. Its core technology is still **guiding the model's behavior through Prompt Engineering**.

1. **Tool Layer (`tools_manager.py`)**:
    - The `scan_table` function is the business process controller for this function. It receives an image containing a table as input.
    - The most critical operation of this function is to **construct a well-defined Prompt**. This Prompt directly instructs the large model to perform two tasks: 1. Identify the table in the image. 2. Return the identified content in Markdown format. This mandatory requirement for the output format is the key to achieving the conversion from unstructured to structured.
    - After constructing the Prompt, it calls the `infer_with_image` method of the model interface layer, passing the image and this formatted instruction together.
    - After receiving the returned Markdown text from the model interface layer, it will perform a file operation: write this text content into a new `.md` file.
    - Finally, it returns structured data containing the new file path and table content.
2. **Model Interface Layer (`large_model_interface.py`)**:
    - The `infer_with_image` function continues to serve as a unified "dispatch center." It receives the image and Prompt from `scan_table` and dispatches the task to the correct backend model implementation based on the current system configuration (`self.llm_platform`).

- Regardless of the backend model, the task of this layer is to handle the communication details with the specific platform, ensure that the image and text data are sent correctly, and then return the plain text returned by the model (in this case, Markdown formatted text) to the tool layer.

In summary, the general flow of table scanning is: `ToolsManager` receives an image and constructs an instruction to "convert the table in this image to Markdown" -> `ToolsManager` calls the model interface -> `model_interface` packages the image and the instruction and sends them to the corresponding model platform according to the configuration -> the model returns Markdown-formatted text -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` saves the text as a `.md` file and returns the result. This process demonstrates how to use the format-following capabilities of a large model to use it as a powerful OCR (Optical Character Recognition) and data structuring tool.

3. Practical Operation

3.1 Configure Offline Large Model

3.1.1 Configure LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. **Open the file in the terminal**:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/Confirm** `llm_platform`:

```
model_service:                          # Model server node parameters
  ros__parameters:
    language: 'en'                      # Large model interface language
    useolinetts: True                   # This item is invalid in text mode
and can be ignored

    # Large model configuration
    llm_platform: 'ollama'              # Key: Make sure this is 'ollama'
    regional_setting : "China"
    camera_type: 'csi'                 # Camera type: 'csi', 'usb'
```

3.1.2 Configure Model Interface (`large_model_interface.yaml`)

This file defines which vision model to use when the platform is selected as `ollama`.

1. Open the file in the terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama-related configuration

```
#.....
## Offline Large Language Models
# Ollama configuration
ollama_host: "http://localhost:11434"  # Ollama server address
ollama_model: "llava"            # Key: Change this to your downloaded multimodal
model, e.g., "llava"
#.....
```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Start and Test the Function (Text Input Mode)

Note: Using a model with a small number of parameters or running with limited memory will result in poor performance. For a better experience, please refer to the corresponding chapter in <Online Large Model (Text Interaction)>.

1. **Prepare the table image file**:

   Place a table image file to be tested in the following path: `~/yahboom_ws/src/largemodel/resources_file/scan_table`

   Then name the image `test_table.jpg`

2. **Start the `largemodel` main program (text mode)**:

   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

3. **Send a text command**:
   Open another terminal and run the following command:

   ```
   ros2 run text_chat text_chat
   ```

   Then start typing: "analyze the table".

4. **Observe the results**:
   In the first terminal running the main program, you will see log output indicating that the system has received the command, called the `scan_table` tool, prompted that scan_table has been executed, and saved the scanned information to the document.

   We can find this document in the ~/yahboom_ws/src/largemodel/resources_file/scan_table path.

4. Common Problems and Solutions

Problem 1: The table content is not fully recognized or contains typos.

Solution:

1. **Image quality**: Ensure that the input table image is clear, not distorted, and evenly lit. Low-quality images are the most common cause of recognition errors.
2. **Model selection**: Models with a larger number of parameters will have better recognition effects. Try switching to a more powerful recognition model.

Problem 2: "Table file not found" is prompted.

Solution:

1. **Check the path**: Make sure you have placed the table image file in the specified path, named it `test_table.jpg`, and have permission to read the file.
2. **Image format**: Make sure the image file is not corrupted and is in .jpg format.